

Testing the Conversational Intent Pipeline API

A practical, no-terminal-required guide to running live demonstrations of the Online Sales intent-routing API for stakeholders and customers.

SERVICE

Online Sales Intent Pipeline — Demo API

INTERFACE

FastAPI · Swagger UI at /docs

AUDIENCE

Engineering manager, prospective customers

STATUS

Testing / demo build — not production

Prepared as a walkthrough for live or recorded demonstrations. Covers: getting the server running, testing through the browser (no curl needed), a scripted multi-turn demo call, what to say out loud while demoing it, and what to be upfront about when asked "is this ready to ship?"

1. What You're Demoing

A customer-support intent engine that reads a free-text message, figures out what the customer wants, pulls out the details it needs (order numbers, dates, discount codes), and tracks the whole conversation across turns — the way a real support call has follow-ups, interruptions, and corrections instead of one message per topic.

It's exposed as an HTTP API with a "call" concept: you open a call, then send it one customer message at a time, and it remembers everything said earlier in that same call. This is what makes it possible to demo realistic multi-turn conversations instead of one-off Q&A.

TALKING POINT FOR THE ROOM

"This isn't a chatbot that guesses — most of the pipeline is deterministic and explainable. It only calls an LLM for the genuinely ambiguous cases: two intents in one sentence, a correction, or a reference back to something said earlier. Everything else is embedding similarity and rule-based extraction, which is why it's fast and consistent."

2. Getting It Running

Do this once, before the room fills up.

1 Install dependencies

```
pip install -r requirements.txt
```

2 Pull the local LLM (only needed the first time — used for disambiguation, not every message)

```
ollama pull gemma3:1b
```

3 Start the server

```
uvicorn server:app --reload
```

Leave this terminal window open and minimized for the duration of the demo — closing it ends every open call.

- 4 **Open the interactive docs** in a browser tab, full-screened for the room:

```
http://127.0.0.1:8000/docs
```

BEFORE YOU'RE ON CAMERA

Run through Section 4's script once, off-camera, right before the demo. It warms up the embedding model and the Ollama connection so the first live request isn't slow, and it confirms Ollama is actually running.

3. Two Ways to Test It

A. Swagger UI — best for the room

The `/docs` page is a full interactive UI generated straight from the API. No terminal, no typing commands live — just forms and an "Execute" button. This is what you want on screen when customers or your manager are watching.

B. curl — best for technical audiences

The exact same requests as raw HTTP calls, for a technically curious customer or manager who wants to see it's "just an API" they could integrate against. Section 5 has the ready-to-paste commands.

4. Swagger UI Walkthrough

With `/docs` open:

- 1 Expand **POST /calls**, click **Try it out**, then **Execute**. The response body returns a `call_id` — copy it. This represents one customer picking up the phone.
- 2 Expand **POST /calls/{call_id}/turns**, click **Try it out**, paste the `call_id` into the path field, and put a customer message in the request body, e.g.:

```
{
  "utterance": "Hi, I wanted to check on my order"
}
```

Click **Execute**. The response is the full decision trace — which intents it considered, the confidence scores, and what it decided to do.

- 3 Send a follow-up turn to the *same* `call_id` — this is the part worth narrating, since it shows the pipeline remembering context:

```
{ "utterance": "It's ORD-48213" }
```

- 4 Expand **GET /calls/{call_id}**, paste the same id, Execute. This shows the accumulated conversation state — every intent detected so far and whether it's still waiting on information.
- 5 Optionally, **DELETE /calls/{call_id}** to close it out cleanly at the end of a demo run.

5. Scripted Demo Call

A realistic multi-turn call that hits every interesting behavior in one continuous conversation: a new intent, a slot fill that fires a tool call, an interruption, and a follow-on request. Run each customer line as one `POST /calls/{call_id}/turns` against the same call.

SPEAKER	MESSAGE	WHAT HAPPENS / WHAT TO POINT OUT
Customer	"Hi, I wanted to check on my order."	NEW_INTENT Matches <code>track_order</code> by meaning, no keyword needed. No order number yet → frame goes to <code>AWAITING_INFO</code> .
Customer	"It's ORD-48213."	SLOT_FILL FIRED Regex pulls the order ID out of a plain sentence. Frame is now complete → a mock <code>get_order_status(order_id='ORD-48213')</code> tool call fires automatically.
Customer	"Actually, while I've got you, can you tell me if the Aurora Desk Lamp is back in stock?"	NEW_INTENT "Actually" trips the correction heuristic, but the new topic (<code>check_product_availability</code>) is confident enough to be treated as an interruption, not a correction – the order-tracking frame is left alone. Product name "Aurora Desk Lamp" is captured by the title-case regex.
Customer	"Also, I think I was charged twice on a different order, ORD-50876."	NEW_INTENT FIRED Third distinct intent (<code>billing_dispute</code>) opened in the same call, order id filled in the same turn it's introduced – fires immediately.

CLOSE THE LOOP

Call `GET /calls/{call_id}` here. It should show **three frames**, all `FIRED`, each with its own resolved tool call — a clean visual of one call handling three unrelated asks without losing track of any of them.

Optional: show the LLM step explicitly

Start a fresh call and send a correction in the same breath as new information, to show the local LLM (Gemma 3B via Ollama) getting invoked and reasoning about it:

```
{ "utterance": "Actually, scratch that, I'd rather exchange it for a different size instead" }
```

The trace's `llm_escalation` field includes the model's one-line reasoning for its decision — worth reading aloud, since it's the one part of the response that isn't just scores and regex matches.

6. Same Script, as curl

For a technical audience, or to leave behind as a copy-pasteable reference.

```
# start a call
curl -s -X POST http://127.0.0.1:8000/calls
# {"call_id":"298cec6767e2"}

# turn 1: new intent, no slot yet
curl -s -X POST http://127.0.0.1:8000/calls/298cec6767e2/turns \
-H "Content-Type: application/json" \
-d '{"utterance":"Hi, I wanted to check on my order"}'

# turn 2: slot fill -> frame FIRED -> tool call
curl -s -X POST http://127.0.0.1:8000/calls/298cec6767e2/turns \
-H "Content-Type: application/json" \
-d '{"utterance":"It''''s ORD-48213"}'

# turn 3: interruption -> new frame, old one untouched
curl -s -X POST http://127.0.0.1:8000/calls/298cec6767e2/turns \
-H "Content-Type: application/json" \
-d '{"utterance":"Actually, can you tell me if the Aurora Desk Lamp is back in stock?"}'

# see everything opened so far
curl -s http://127.0.0.1:8000/calls/298cec6767e2

# tear down at the end of the demo
curl -s -X DELETE http://127.0.0.1:8000/calls/298cec6767e2
```

7. What to Highlight, Turn by Turn

- ✓ **Speed & cost:** most turns never touch the LLM — embedding similarity plus regex resolves the majority of messages, so it's cheap and fast at volume.
- ✓ **Explainability:** every response includes the candidate intents and their confidence scores, not just a final answer — useful for a customer asking "how do we know it got this right?"
- ✓ **Memory across turns:** the order id given in turn 2 is used automatically; the customer never has to repeat context, and unrelated topics raised mid-call don't get lost or merge into each other.
- ✓ **Selective LLM use:** the model is only invoked for genuinely ambiguous language (corrections, references, multi-intent, close scoring calls) — matching the design principle of using the cheapest reliable method first.
- ✓ **Tool-call shape:** a completed frame resolves to a named action with typed arguments (`get_order_status(order_id=...)`), the same shape a real backend integration would consume.

8. What to Say If Asked “Is This Production-Ready?”

BE UPFRONT — THIS IS A TESTING/DEMO BUILD

This API is explicitly a demo/testing harness, not a production service. It's worth stating plainly rather than letting it come up as a surprise later:

- **State is in-memory only.** A server restart loses every open call; there's no database or Redis behind it yet.
- **Single process only.** It can't run behind a load balancer with multiple workers as-is — each would have its own empty state.
- **No authentication.** Anyone who can reach the port can start, read, or delete any call. Fine on localhost for a demo; not exposed past that.
- **No automatic cleanup.** Calls accumulate in memory until the process restarts; there's no TTL or expiry yet.

Frame it as: *the intent pipeline itself — retrieval, classification, slot extraction, dialogue state, LLM escalation — is the real engineering; the HTTP layer around it is a thin demo harness, and productionizing it is a scoped, well-understood next step (state store → Redis, add auth, multi-worker support).*

9. Endpoint Quick Reference

METHOD & PATH	BODY	RETURNS
POST /calls	—	Starts a new call → {"call_id": "..."}
POST /calls/{call_id}/turns	{"utterance": "..."}	Full decision trace for that turn
GET /calls/{call_id}	—	Every frame opened in the call so far
DELETE /calls/{call_id}	—	204, drops the call's state
GET /health	—	{"status": "ok"}

An unknown `call_id` returns `404` on any `/calls/{call_id}...` route. Full interactive reference always available live at `/docs`.